UNIVERSIDAD NACIONAL DE COLOMBIA	Sede Bogotá Facultad de Ingeniería Departamento de Sistemas e Industrial Ingeniería de Software I (2016701)

Proyecto

Lineamientos generales

1. Naturaleza del proyecto: vale el 70% de la nota y se desarrolla durante todo el semestre.
Integra los temas del curso: ingeniería de requerimientos, fundamentos de diseño, testing y arquitectura de software. No es un proyecto por contrato: es un producto. Nadie les va a decir qué construir. El equipo lo decide, y tiene que resolverle un problema real a alguien que no está obligado a usarlo.
Modo de ejecución: servicio.
La aplicación corre del lado del proveedor, no se instala en la máquina del usuario.

2. Proceso de desarrollo Desarrollo iterativo e incremental. Cada entrega es un incremento funcional: software que corre, no documentos sobre software que correrá. No se acepta cascada disfrazada de iteraciones. Prácticas exigidas:

- Repositorio Git desde el primer día, con historial real: cada integrante hace sus propios commits bajo su identidad.
- Pruebas unitarias de la lógica de negocio, con cobertura mínima del 80% sobre esas clases. Deben correr sin base de datos ni servidor.
- Revisión entre pares: todo cambio entra a la rama principal por pull request aprobado por otro integrante. Sin commits directos a la principal.
- README verificable: alguien ajeno al equipo debe poder clonar el repositorio y levantar la aplicación siguiendo el README, sin ayuda de ustedes.

3. Restricciones técnicas Arquitectura monolítica. Un repositorio, un artefacto desplegable, un proceso. Sin microservicios ni arquitecturas distribuidas. Sin frontend separado (SPA) ni API REST usada como frontera interna. La interfaz se sirve con vistas renderizadas en el servidor.

Stack obligatorio, igual para todos los equipos:

Elemento	Definición
Lenguaje	Java 21 (LTS)
Framework	Spring Boot con servidor embebido

Vistas	Thymeleaf
Construcción	Maven, con el wrapper (./mvnw) incluido en el repositorio
Base de datos	Relacional embebida en archivo (H2 o SQLite)

La aplicación debe levantarse con un solo comando documentado: sin instalar servidores, sin configurar variables de entorno, sin pasos manuales previos. PostgreSQL se acepta como alternativa, siempre que se mantenga esa condición.
Lógica de negocio aislada.
Las reglas del negocio deben poder probarse sin levantar la aplicación ni la base de datos. Es un criterio verificable y se califica. Base de datos. Esquema versionado en el repositorio. Datos semilla suficientes para ejercitar las reglas de verdad: una demostración con tres registros esconde todos los errores. APIs externas. Se permite una sola, con tres condiciones: que no sea el núcleo del producto (la aplicación debe seguir siendo útil sin ella), que esté detrás de un adaptador propio con una implementación falsa para las pruebas, y que se justifique por escrito.

4. Tema del proyecto:
El problema debe ser realista y acotado, con lógica de negocio clara. En la propuesta responden explícitamente: ¿Qué decide su software que hoy un humano tendría que decidir a mano? Si la respuesta es "nada", la propuesta se rechaza. Guardar, listar, editar y borrar registros no es lógica de negocio: es almacenamiento. Lo que se califica son las reglas - asignación, priorización, cálculo, validación cruzada, cambios de estado. Acceso a usuarios. El problema debe estar en un contexto donde el equipo pueda hablar con al menos un usuario potencial real: compañeros, profesores, la administración de la facultad, un negocio del barrio, una organización a la que pertenezcan. Se aceptan versiones propias de aplicaciones que ya existen, siempre que el alcance esté bien acotado y pasen el filtro de arriba. Quedan descartados: videojuegos; temas ya aprobados a otro equipo (por orden de llegada); partir de un repositorio, un tutorial o un proyecto de semestre anterior.

5. Equipos de cinco integrantes.
La conformación se fija en la Entrega 1 y no cambia, salvo retiro formal de la asignatura, que el equipo informa por escrito para que se ajuste el alcance esperado.

6. Entregas

Entrega	Contenido	Semana	Peso
E1	Propuesta de producto y requerimientos	4	10%

E2	Incremento 1	10	30%
E3	Incremento 2 y cierre	16	30%

(Pesos sobre la nota total del curso; suman el 70% del proyecto). E1 Propuesta de producto (documento escrito): el problema y a quién le duele; evidencia de al menos una entrevista con un usuario potencial real (a quién, cuándo, qué dijo, qué cambió en su idea después); la respuesta a la pregunta de la sección 4; el alcance, incluyendo qué no van a construir; y quién pagaría por esto y por qué, aunque sea en una línea. Aquí no se pide arquitectura. E2 - Primer incremento: software que resuelve de punta a punta al menos una regla de negocio, aunque sea la más simple del producto; documento de arquitectura; pruebas de la lógica de negocio; README verificable. E3: el alcance de este incremento final lo propone el equipo y se acuerda al inicio de la iteración. Los criterios se comunican con antelación.

7. Defensas individuales Hay defensa en cada entrega. Se seleccionan al azar integrantes de cada equipo para responder individualmente por lo entregado, incluido código que no escribieron. En la entrega final se evalúa a todos. La nota de la defensa modula la nota grupal de esa entrega: es responsabilidad colectiva del equipo que todos sus integrantes entiendan el producto.

8. Uso de asistentes de IA Permitido, y debe declararse en una sección corta del README: qué herramientas, en qué partes, para qué. No hay penalización por usarlos. Sí la hay por no poder explicar lo entregado. La defensa individual es el filtro.

9. Documentación mínima README (qué es el producto, cómo levantarlo, cómo correr las pruebas, declaración de uso de IA) y documento de arquitectura (capas, modelo de dominio, dependencias entre componentes).

10. Despliegue Opcional. Si lo intentan, usen créditos gratuitos y no comprometan dinero propio. Ningún criterio de calificación depende de que el despliegue funcione.

11. Propiedad del trabajo El trabajo es de ustedes. El curso no reclama propiedad sobre lo que construyan.